Vulnerable Bank Report

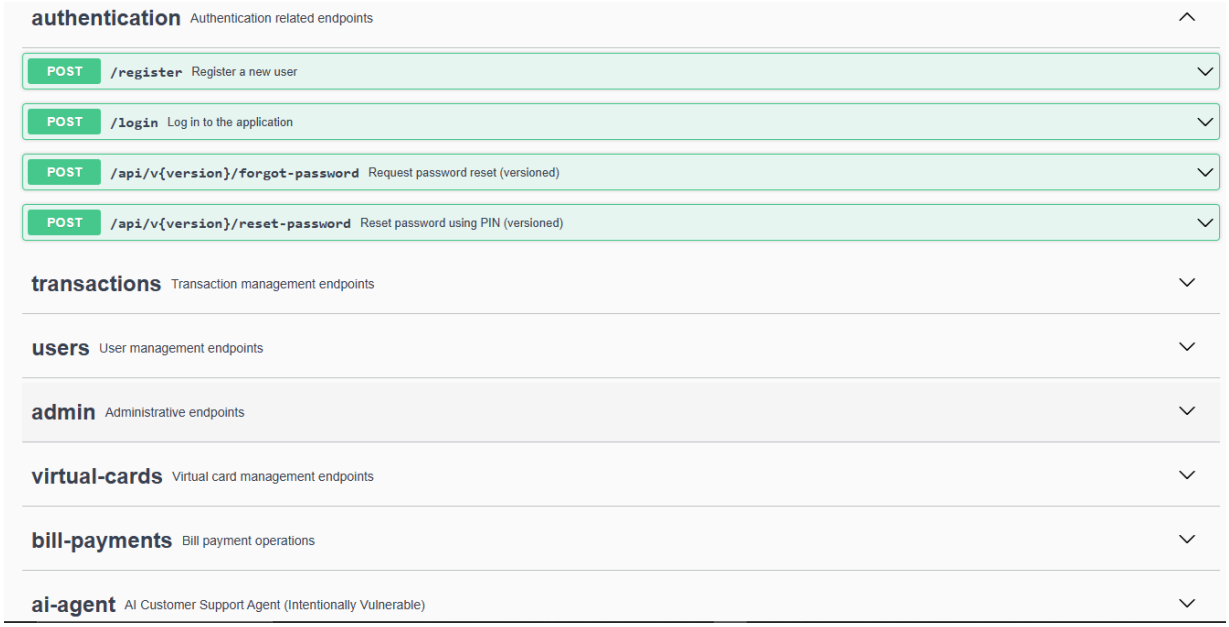
Tester: Charmaine Mangorima
Date: December 3, 2025
Test Environment: Vulnerable Bank running on http://localhost:5000.
Tools Used: Postman, Cursor(to start server), Burpsuite, Swagger Editor, xjwt.io, oauth.tools.

Executive summary

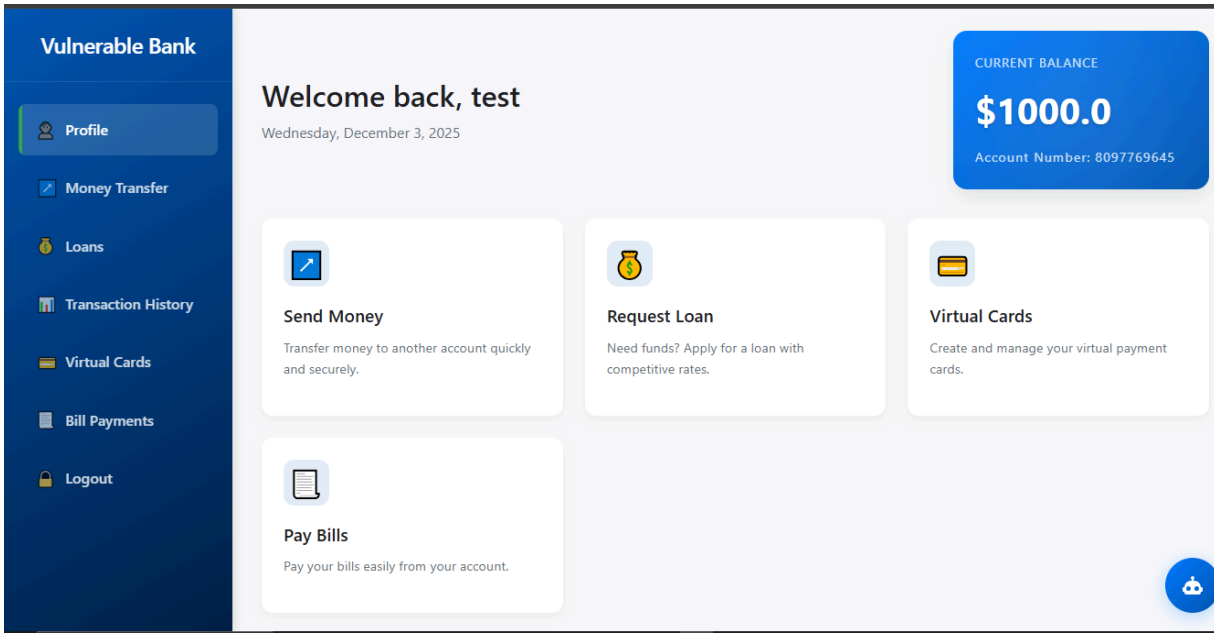
This report summarizes an assessment of three major API vulnerabilities in the Vulnerable Bank application: Broken Object Level Authorization(BOLA), Broken Authentication and Broken Object Property Level Authorization(BOPLA). Testing showed that attackers can access other users' data by altering IDs in API requests, reuse or tamper with session tokens due to weak authentication controls and modify restricted fields such as administrative properties because the server fails to properly filter JSON input. These flaws highlight critical gaps in access control, token management and data validation that expose the application to unauthorized access and data compromise.

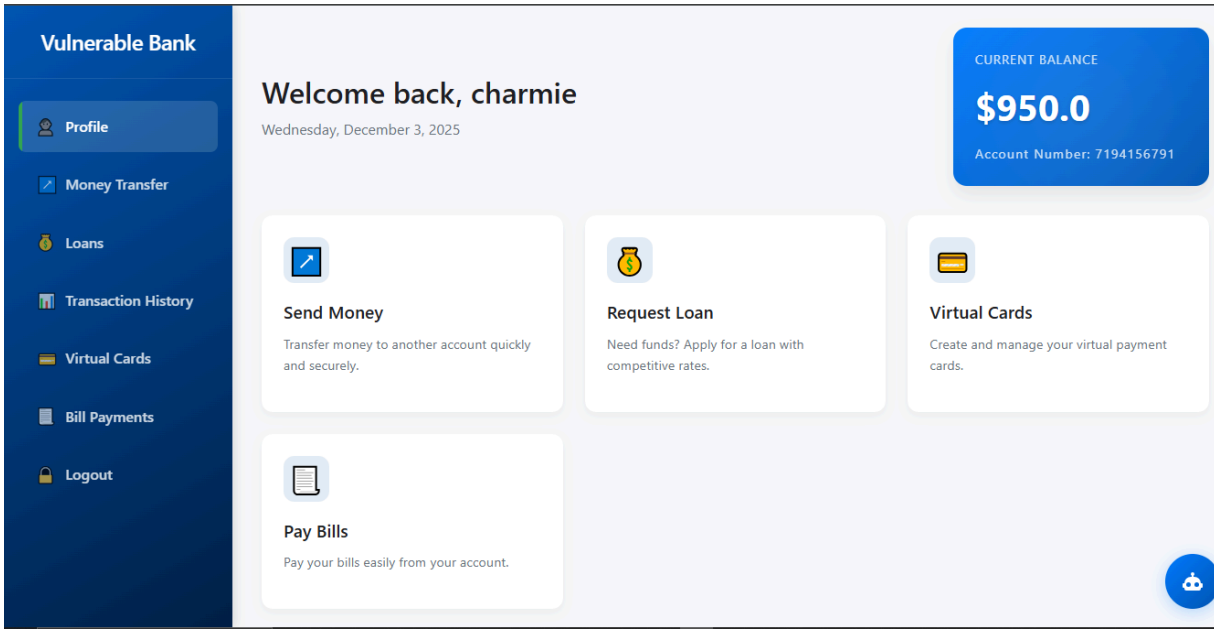
Findings

I used Swagger Editor to get a clear view and understanding of the API endpoints:



I created test user accounts namely test and charmie:



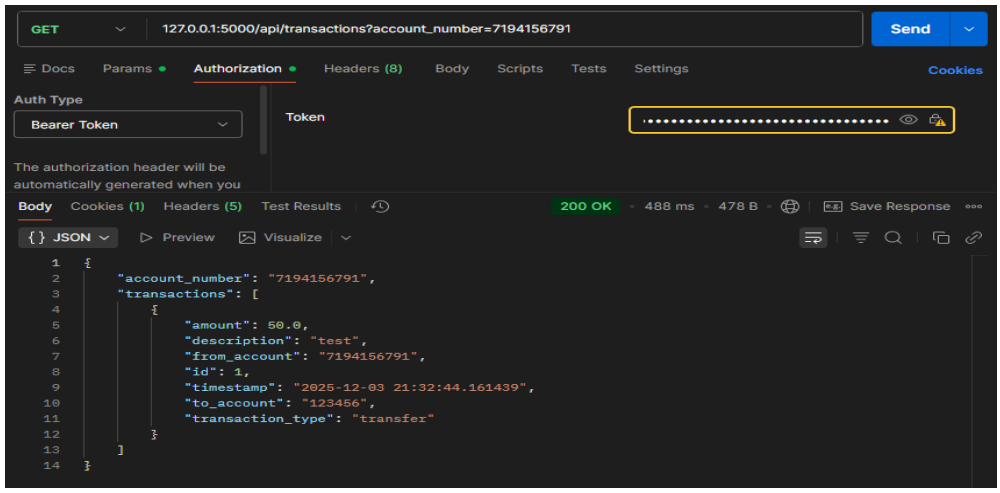


1. **API1 Broken Object Level Authorization**

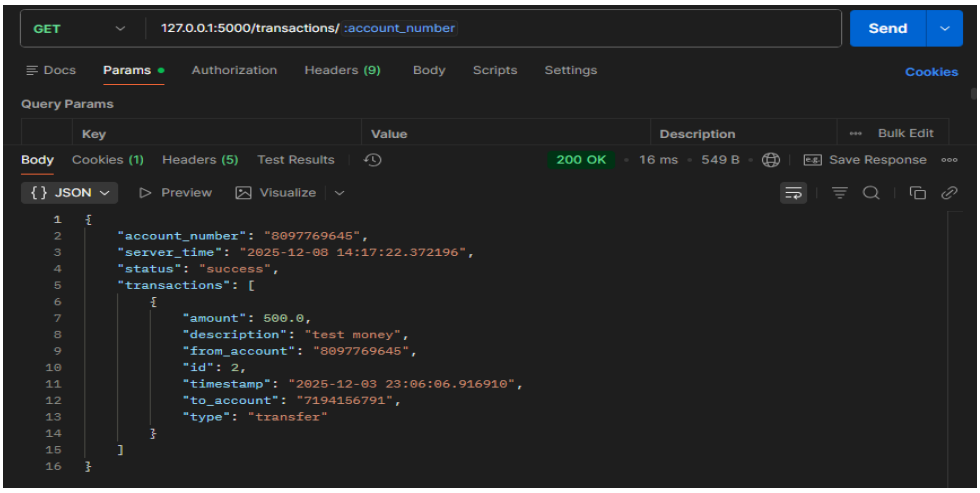
   BOLA is when the API allows user A to access user B's data by manipulating object IDS. In this case, a user is able to alter the account number and access another user's transactions.

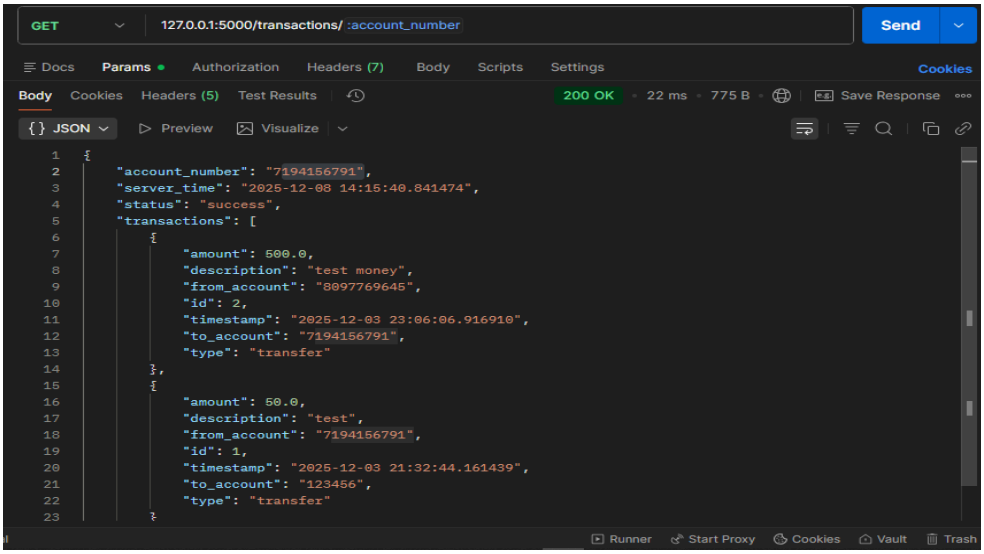
Evidence

1. Captured the ***GET 127.0.0.1:5000/api/transactions?account_number=string*** whilst logged in as test(8097769645) and i managed to use charmie's account number(7194156791) and the request returned the list of charmie's transactions. The API does not check authorization or ownership.

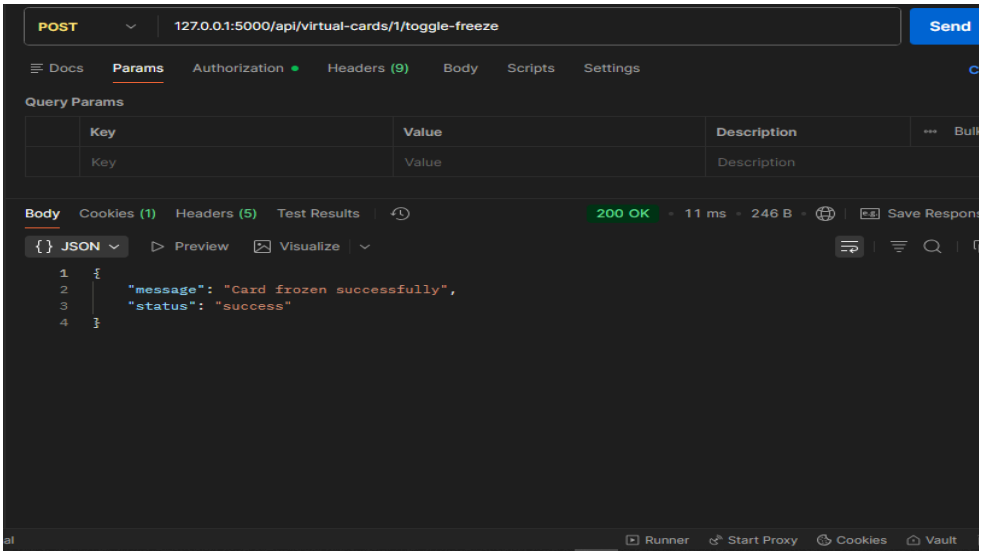


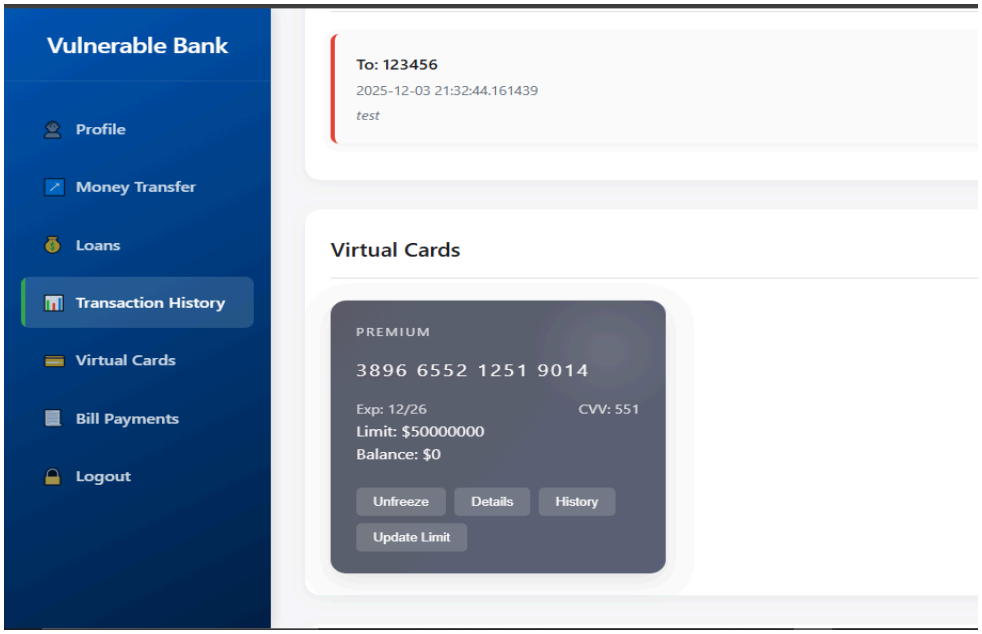
2. Any user can see another user's transactions even without logging in on the ***127.0.0.1:5000/transactions/:account_number***. This represents classic BOLA because there is no authentication or authorization checks to access sensitive data.



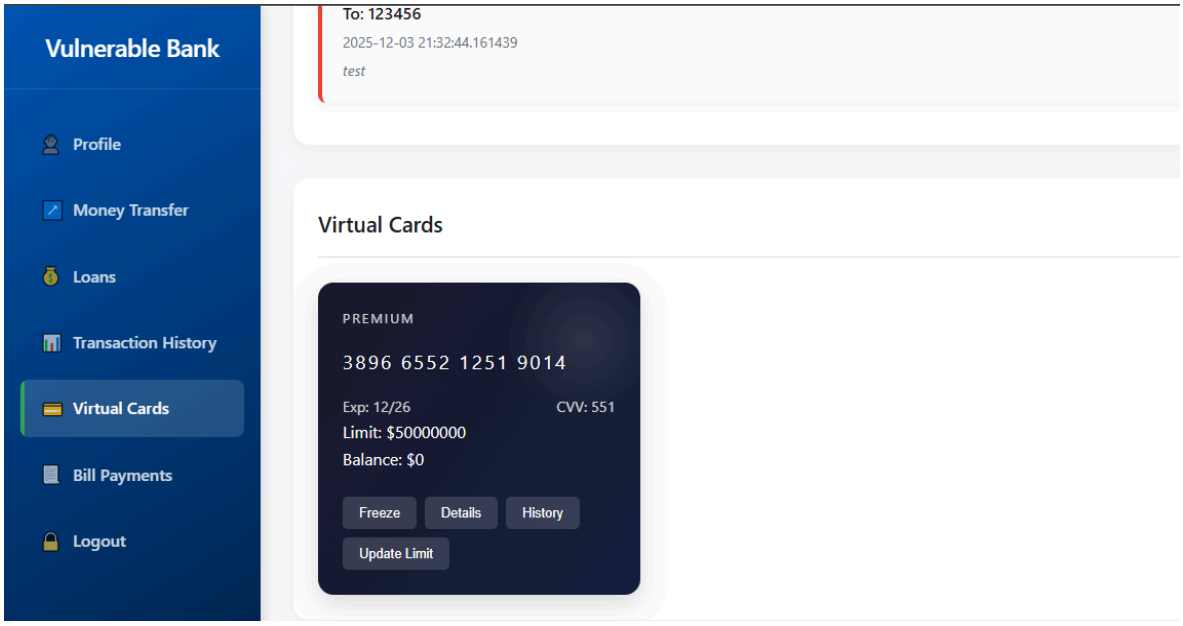


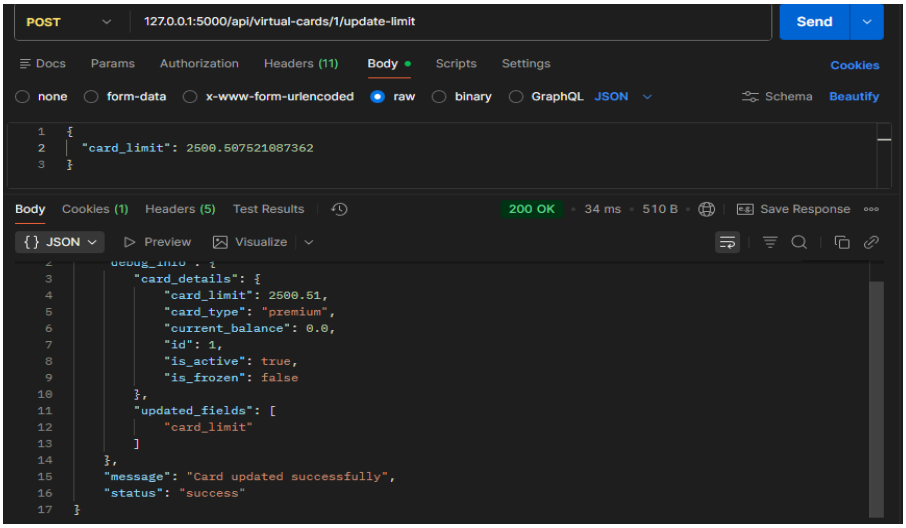
3. I also managed to freeze charmie's virtual card via the ***POST 127.0.0.1:5000/api/virtual-cards/:card_id/toggle-freeze*** whilst logged in as test by changing the card_id in the url.

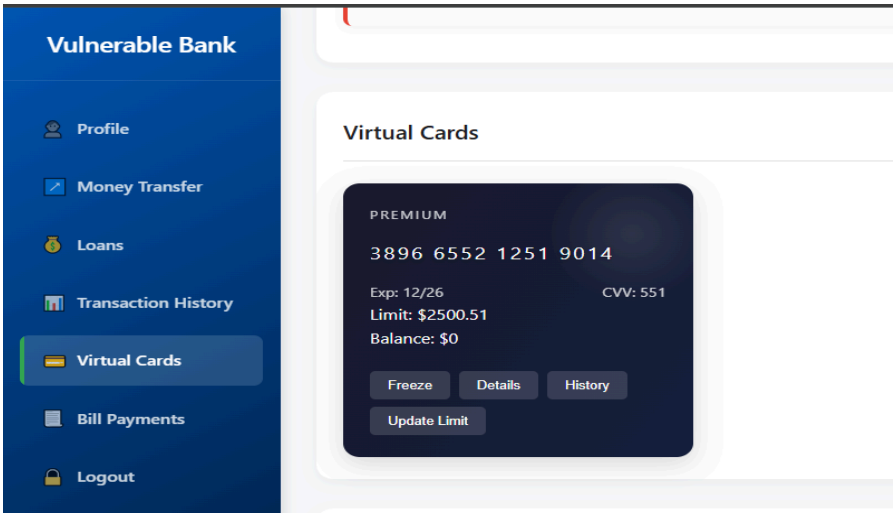




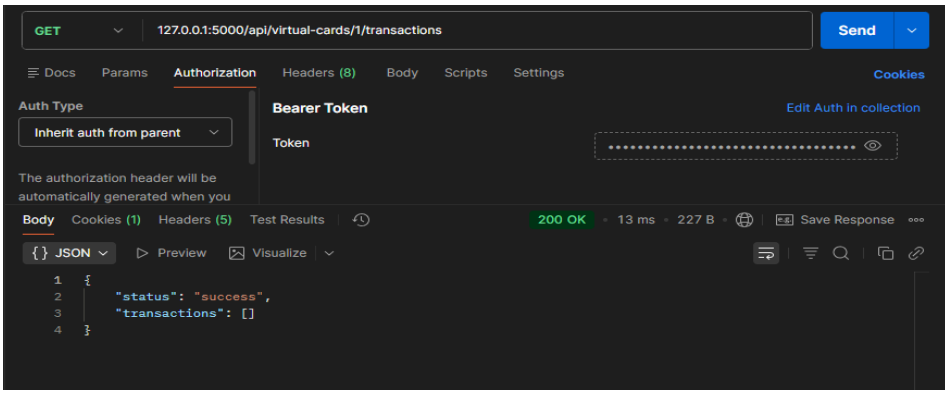
4. The ***POST 127.0.0.1:5000/api/virtual-cards/card_id/update-limit*** endpoint is also vulnerable to BOLA because the test user was able to update charmie's virtual card limit by changing the card_id in the request.



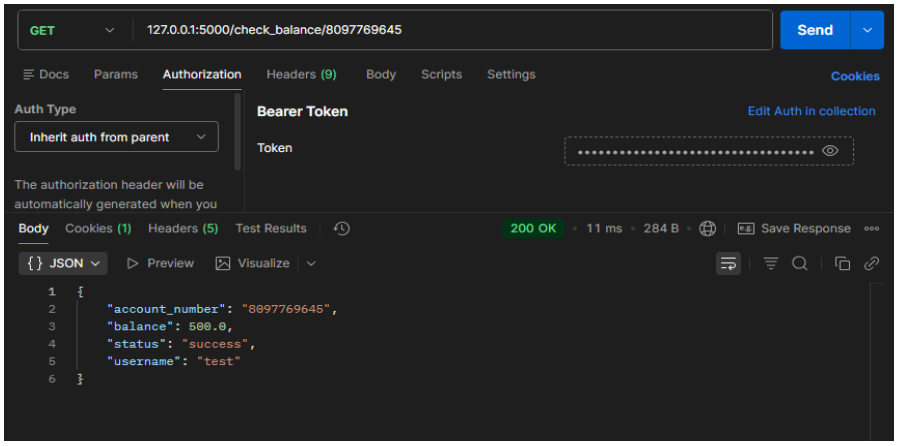


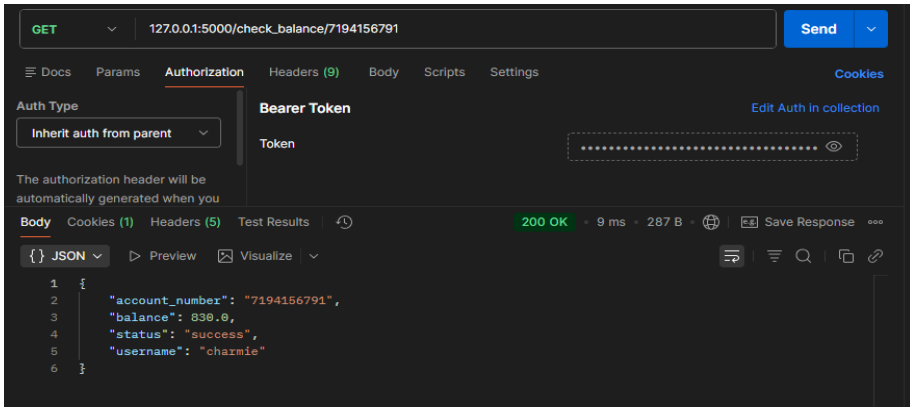


5. The ***GET 127.0.0.1:5000/api/virtual-cards/card_id/transactions*** endpoint is vulnerable to BOLA because the test user was able to access charmie's virtual card transactions by altering the card_id in the request.



6. From the ***GET 127.0.0.1:5000/check_balance/account_number***, the test user was able to access charmie's account balance by changing the cccount_number in the url.





Recommendations

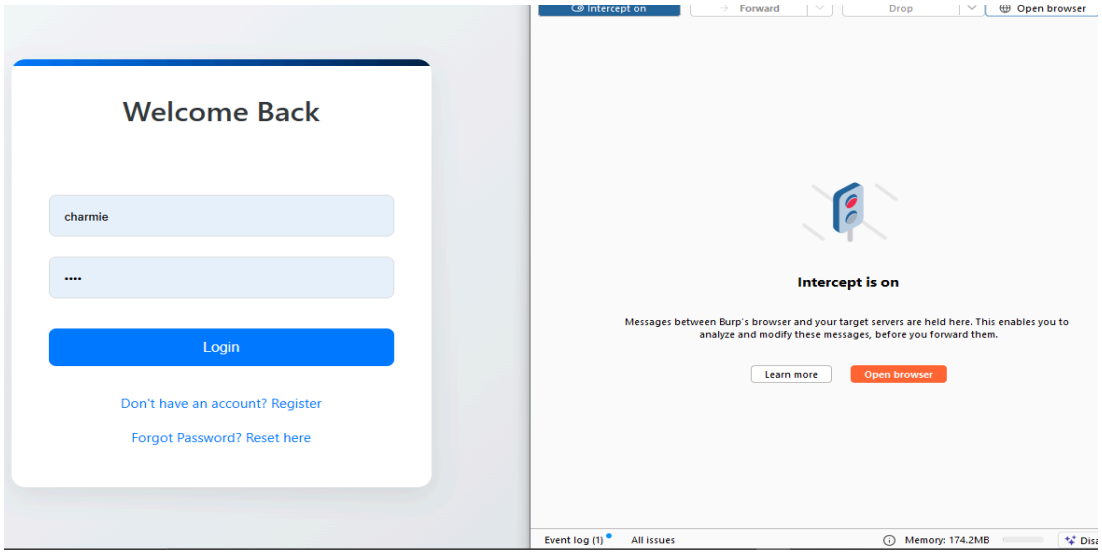
Always validate that the authenticated user is authorized to access or modify the data. Implement authentication before authorization.

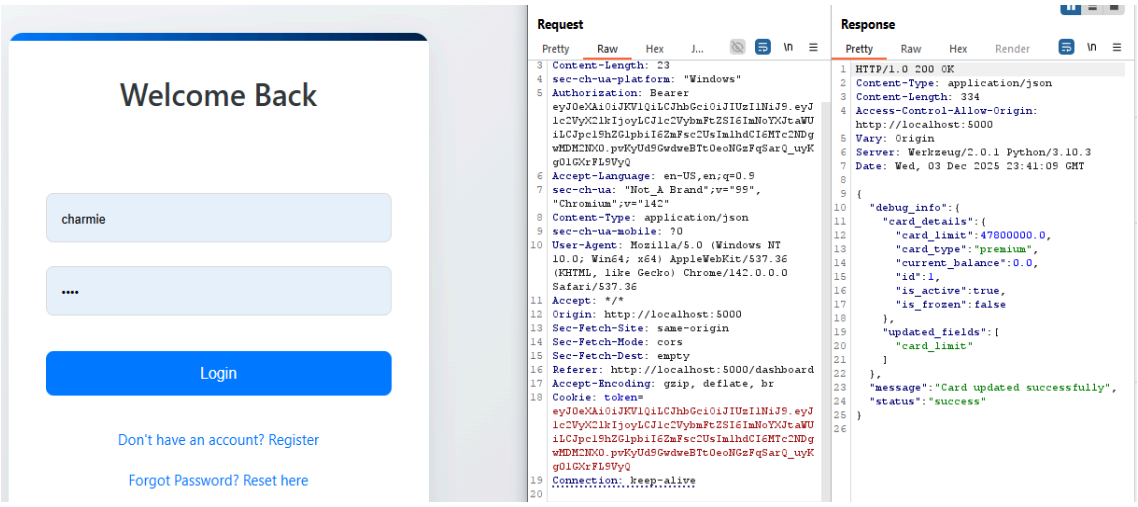
2. API2 Broken Authentication

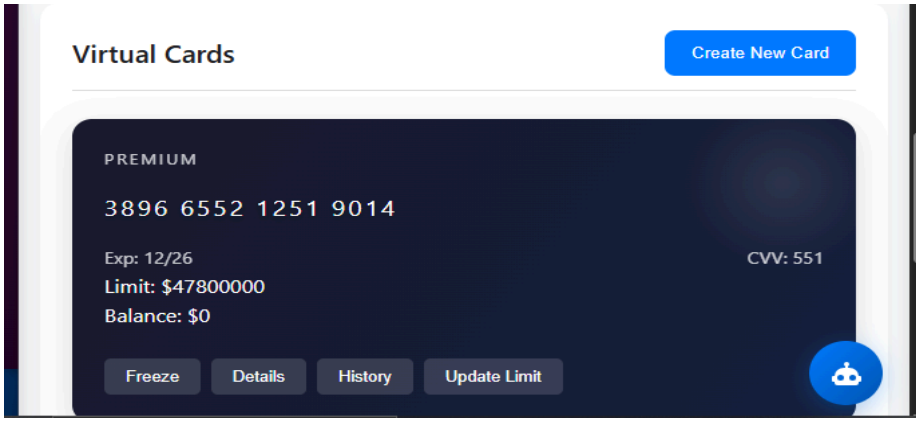
Broken authentication is when the API has flaws in the authentication process that may result in users manipulating tokens, bruteforcing weak passwords or reusing old tokens to authenticate.

Evidence

I was logged in as charmie and then i logged out. The token is expected to become invalid after logging out but this one was always valid. In Burpsuite, I was able to update charmie's card limit using the old token and the API updated the limit successfully. Upon inspection, I noticed that the JWT does not have an expiry claim in the payload.







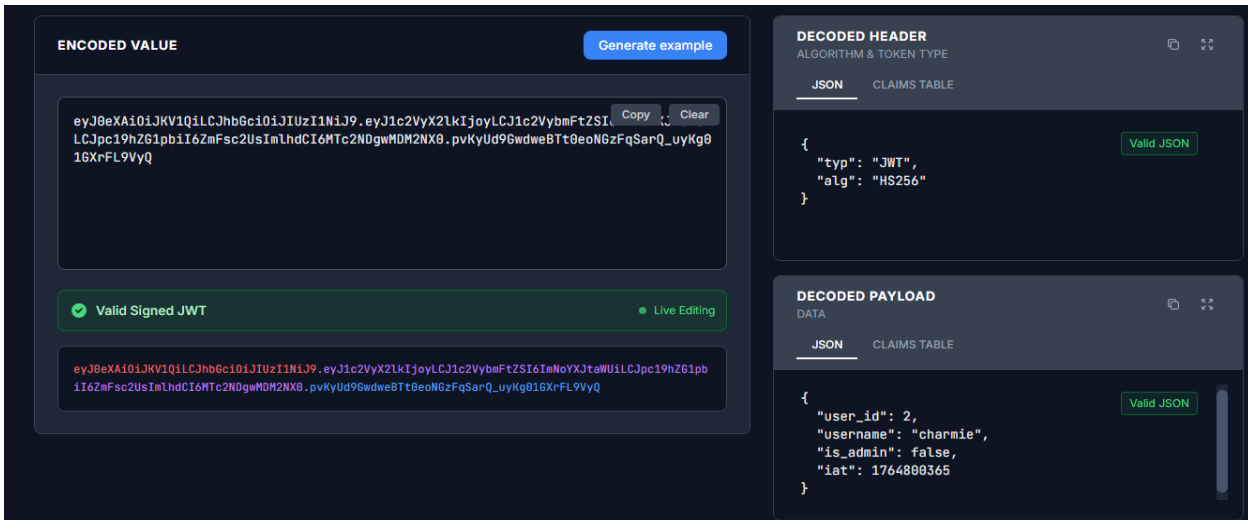
Using xjwt.io, I cracked the old jwt which had a weak signing algorithm(HS256) and I managed to create a new JWT with escalated privileges. Using this new admin token I managed to create an admin account that is a request that requires admin privileges.

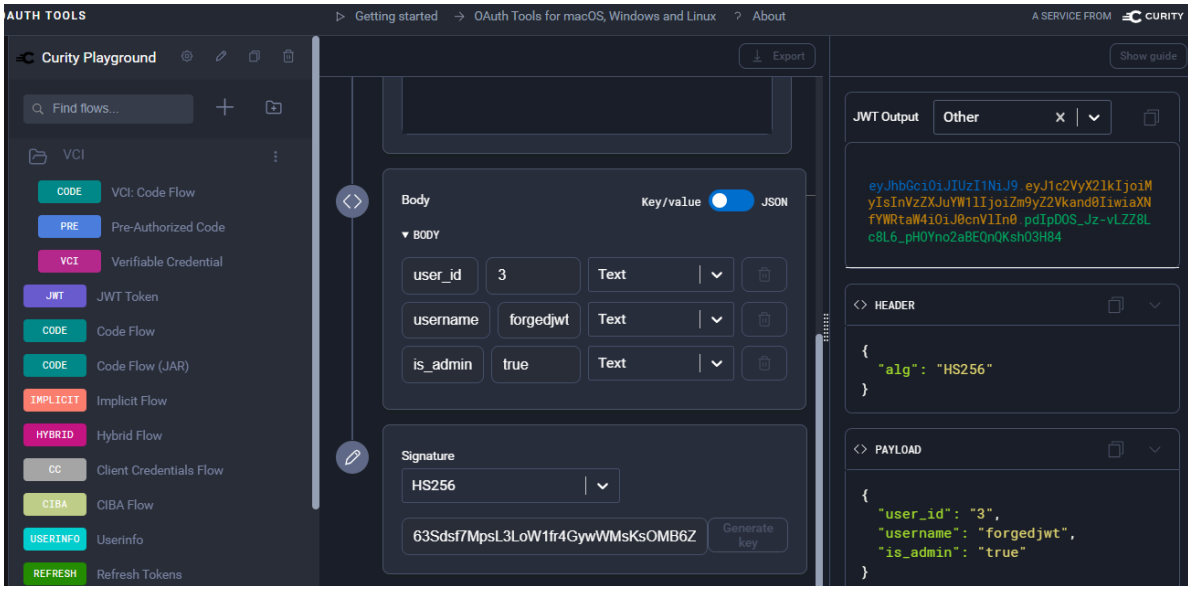
Old JWT:

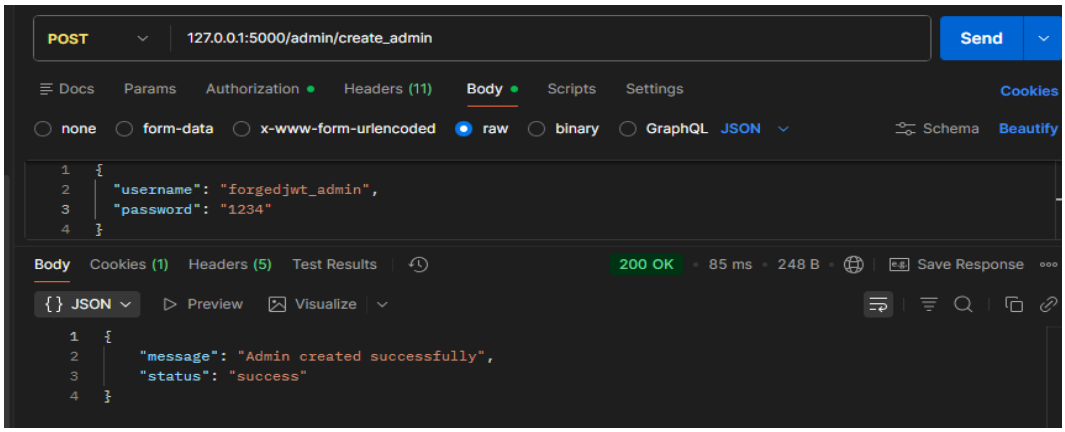
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6ImNoYXJtaWUiLCJpc19hZG1pbiI6ZmFsc2UsImlhdCI6MTc2NDgwMDM2NX0.pvKyUd9GwdweBTt0eoNGzFqSarQ_uyKg01GXrFL9VyQ

New JWT:

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoiMyIsInVzZXJuYW1lIjoiZm9yZ2Vkand0IiwiaXNfYWRtaW4iOiJ0cnVlIn0.pdIpDOS_Jz-vLZZ8Lc8L6_pHOYno2aBEQnQKshO3H84








```python
def verify_token(token):
    """
    Verify JWT token with multiple vulnerabilities
    - Accepts 'none' algorithm (CWE-347)
    - No signature verification in some cases
    - No expiration check
    """
    try:
        # Vulnerability: Accepts any algorithm, including 'none'
        payload = jwt.decode(token, JWT_SECRET, algorithms=ALGORITHMS)
        return payload
    except jwt.exceptions.InvalidSignatureError:
        # Vulnerability: Still accepts tokens in some error cases
        try:
            # Second try without verification
            payload = jwt.decode(token, options={'verify_signature': False})
            return payload
        except:
            return None
    except Exception as e:
        # Vulnerability: Detailed error exposure in logs
        print(f"Token verification error: {str(e)}")
        return None
```

In the code, the *verify_token* function does not check token expiration as shown above. This is why an expired JWT is still accepted.

Recommendations

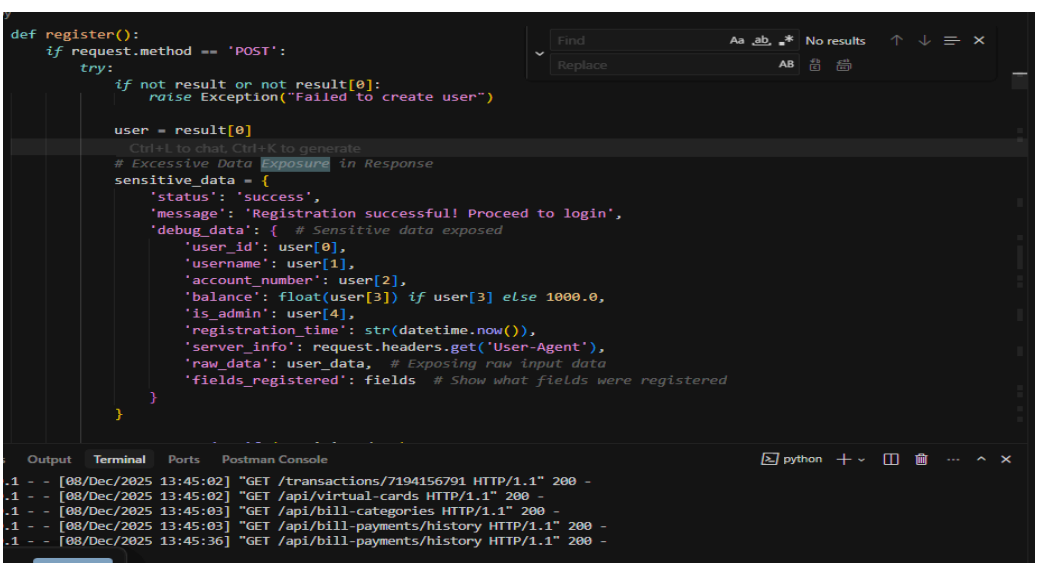
Implement strong password policies
Use strong algorithms to secure tokens
Make sure that tokens have validity claims such as the expiry time
Implement multi-factor authentication (MFA).

3. API3 Broken Object Property Level Authorization

BOPLA is when the API exposes too much information in the response including sensitive information that is not necessary to the request. It is also when an API allows users to edit sensitive fields or add extra data in their request payload resulting in privilege escalation.

Evidence

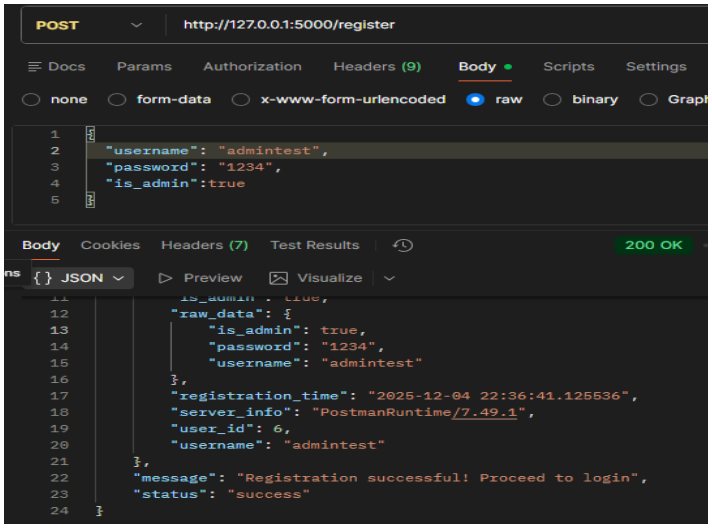
1. The ***POST /register*** endpoint does not check for mass assignment and it also exposes excessive data in the response after registering a user. A user is able to register a new account and add ***is_admin:true*** to the payload (privilege escalation).



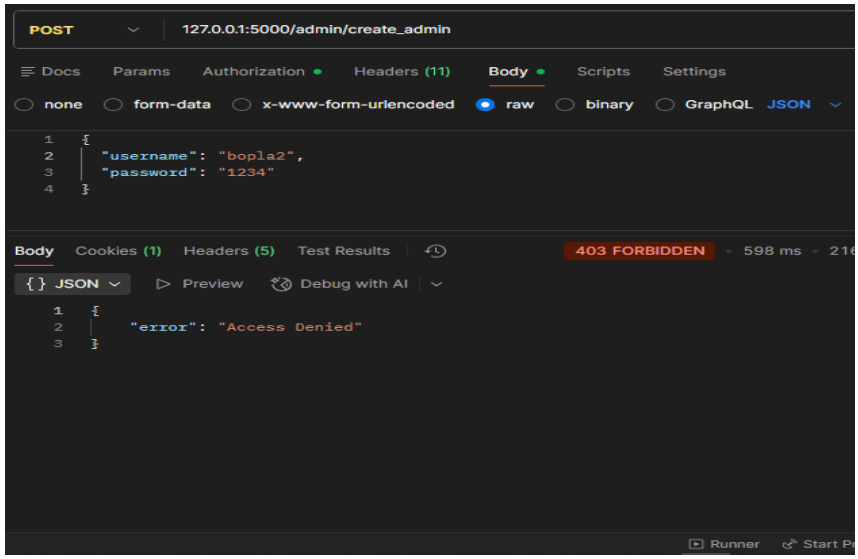
```
def register():
    if request.method == 'POST':
        try:
            if not result or not result[0]:
                raise Exception("Failed to create user")

            user = result[0]
              Ctrl+L to chat, Ctrl+K to generate
            # Excessive Data Exposure in Response
            sensitive_data = {
                'status': 'success',
                'message': 'Registration successful! Proceed to login',
                'debug_data': {  # Sensitive data exposed
                    'user_id': user[0],
                    'username': user[1],
                    'account_number': user[2],
                    'balance': float(user[3]) if user[3] else 1000.0,
                    'is_admin': user[4],
                    'registration_time': str(datetime.now()),
                    'server_info': request.headers.get('User-Agent'),
                    'raw_data': user_data,  # Exposing raw input data
                    'fields_registered': fields  # Show what fields were registered
                }
            }
```

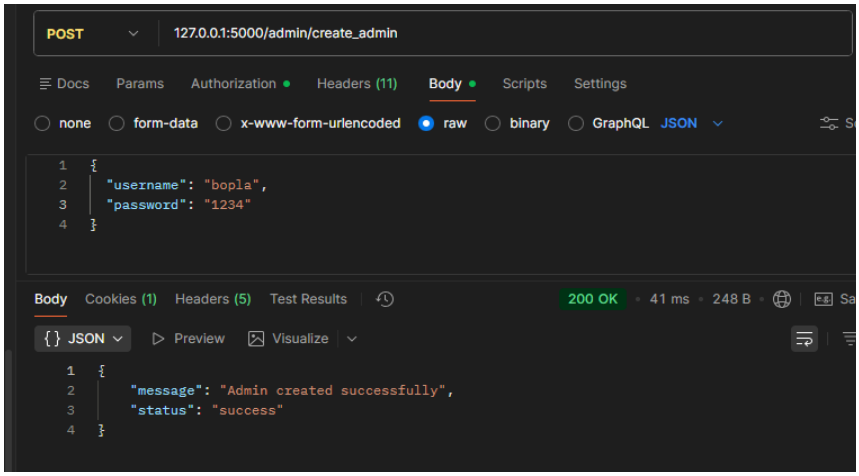
```
Output   Terminal   Ports   Postman Console
.1 - - [08/Dec/2025 13:45:02] "GET /transactions/7194156791 HTTP/1.1" 200 -
.1 - - [08/Dec/2025 13:45:02] "GET /api/virtual-cards HTTP/1.1" 200 -
.1 - - [08/Dec/2025 13:45:03] "GET /api/bill-categories HTTP/1.1" 200 -
.1 - - [08/Dec/2025 13:45:03] "GET /api/bill-payments/history HTTP/1.1" 200 -
.1 - - [08/Dec/2025 13:45:36] "GET /api/bill-payments/history HTTP/1.1" 200 -
```



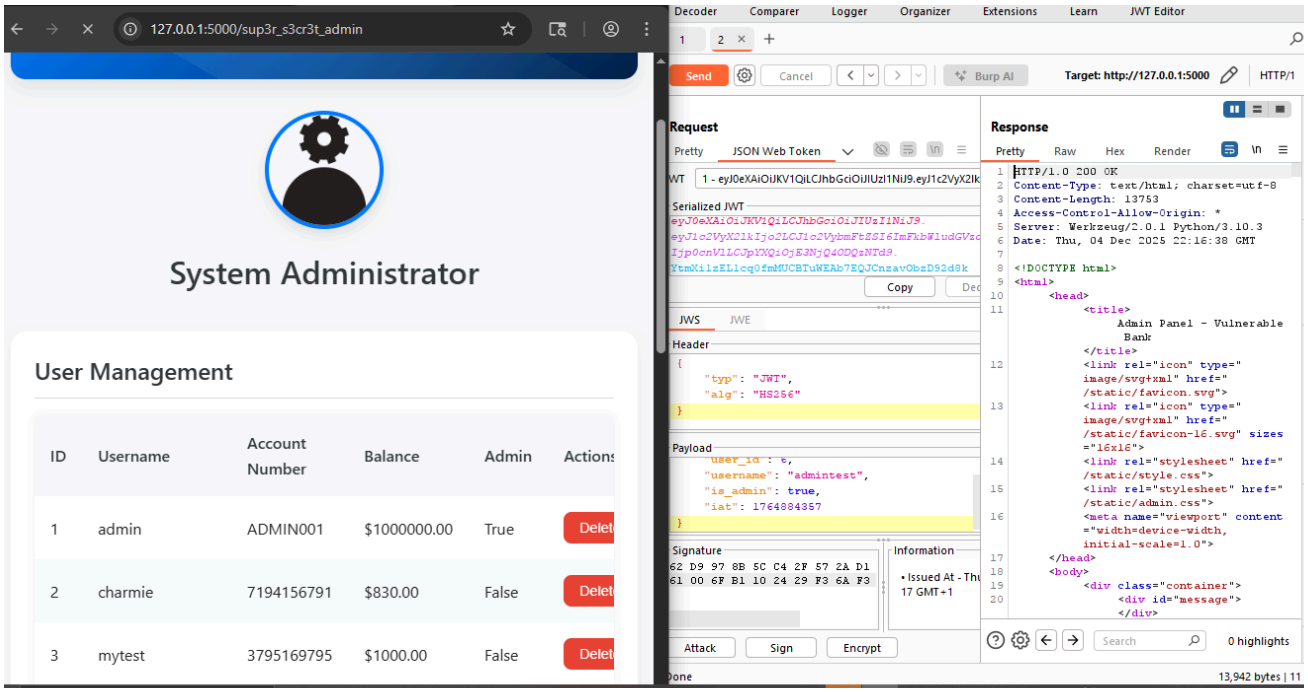
To confirm that I successfully registered an admin, I tested on the ***admin/create_admin*** endpoint which requires admin privileges. I tried it with a regular user's account and it did not work.



I then used the newly created admin account and it worked.

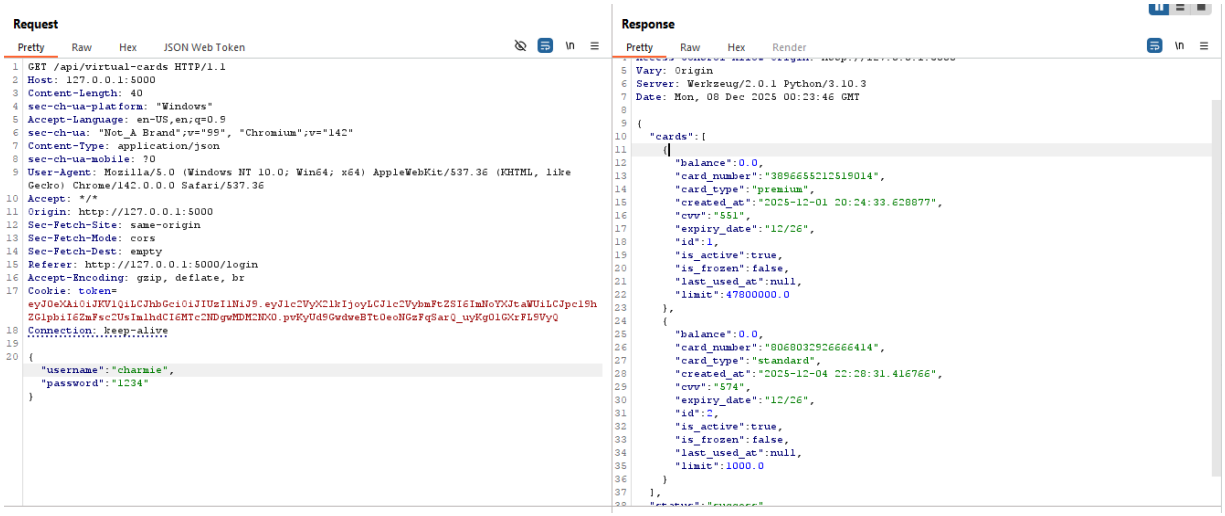


Using the admin account I was able to access the secret admin dashboard. This way the attacker will have full access to all user data and accounts.



2. The ***GET /api/virtual_cards*** endpoint is vulnerable to Broken Object Property Level Authorization because it returns sensitive object properties that should never be exposed to the client, even when the user is authenticated. The API discloses full card numbers, CVVs, expiration dates, limits, and internal flags such as is_active and is_frozen, none of which should be readable or retrievable from the client side. Proper property-level authorization would restrict access to only non-sensitive, user-safe fields,typically masked card numbers and minimal metadata,while preventing exposure of internal or high-risk attributes. By failing to filter or sanitize the object properties

before sending the response, the application leaks confidential data and reveals internal fields that should remain server-side only, demonstrating a classic BOPLA flaw.



Request

Pretty Raw Hex JSON Web Token

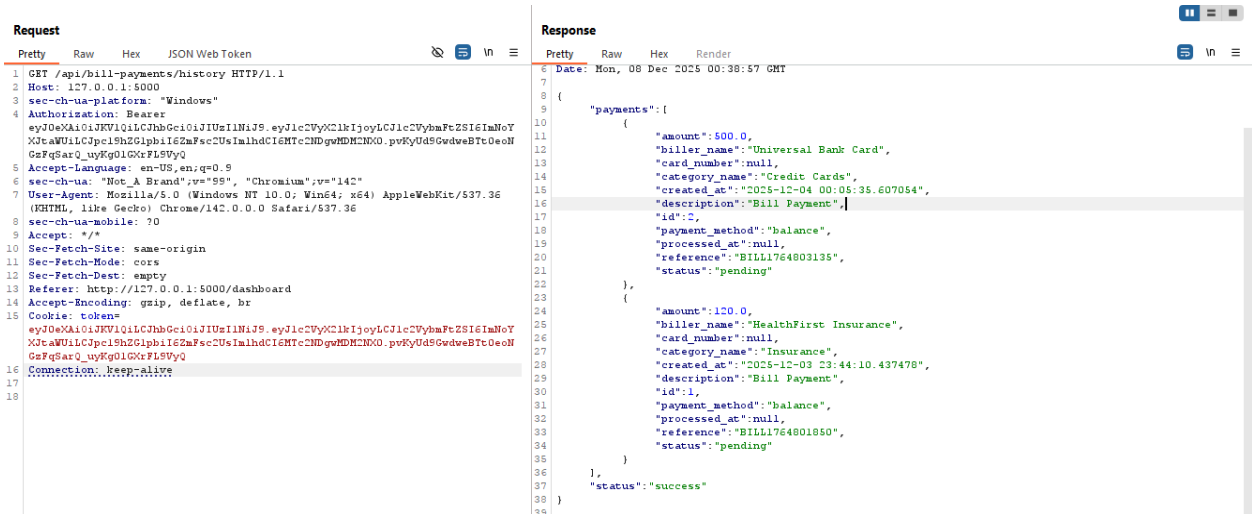
```
1  GET /api/virtual-cards HTTP/1.1
2  Host: 127.0.0.1:5000
3  Content-Length: 40
4  sec-ch-ua-platform: "Windows"
5  Accept-Language: en-US,en;q=0.9
6  sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
7  Content-Type: application/json
8  sec-ch-ua-mobile: ?0
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
   Gecko) Chrome/142.0.0.0 Safari/537.36
10 Accept: */*
11 Origin: http://127.0.0.1:5000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://127.0.0.1:5000/login
16 Accept-Encoding: gzip, deflate, br
17 Cookie: token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoyLCJlc2VybmFtZSI6ImNoYXJtaWUiLCJpcl9h
   ZGlpbiI6ZmFsc2UsImlhdCI6MTc2NDgwMDM2NX0.pvKyUd9GwdweBTt0eoNGzFqSarQ_uyKg01GXrFL9VyQ
18 Connection: keep-alive
19
20 {
     "username":"charmie",
     "password":"1234"
   }
```

Response

Pretty Raw Hex Render

```
5  Vary: Origin
6  Server: Werkzeug/2.0.1 Python/3.10.3
7  Date: Mon, 08 Dec 2025 00:23:46 GMT
8
9  {
10   "cards":[
11     {
12       "balance":0.0,
13       "card_number":"3896655212519014",
14       "card_type":"premium",
15       "created_at":"2025-12-01 20:24:33.628877",
16       "cvv":"551",
17       "expiry_date":"12/26",
18       "id":1,
19       "is_active":true,
20       "is_frozen":false,
21       "last_used_at":null,
22       "limit":47800000.0
23     },
24     {
25       "balance":0.0,
26       "card_number":"8068032926666414",
27       "card_type":"standard",
28       "created_at":"2025-12-04 22:28:31.416766",
29       "cvv":"574",
30       "expiry_date":"12/26",
31       "id":2,
32       "is_active":true,
33       "is_frozen":false,
34       "last_used_at":null,
35       "limit":1000.0
36     }
37   ],
```

3. The ***GET /api/bill-payments/history*** is vulnerable to information disclosure because it is returning sensitive information like the card_number but the ui filters it. Developers should never depend on ui filtering but at API level.



Request

Pretty Raw Hex JSON Web Token

```
1  GET /api/bill-payments/history HTTP/1.1
2  Host: 127.0.0.1:5000
3  sec-ch-ua-platform: "Windows"
4  Authorization: Bearer
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoyLCJlc2VybmFtZSI6ImNoY
   XJtaWUiLCJpcl9hZGlpbiI6ZmFsc2UsImlhdCI6MTc2NDgwMDM2NX0.pvKyUd9GwdweBTt0eoN
   GzFqSarQ_uyKg01GXrFL9VyQ
5  Accept-Language: en-US,en;q=0.9
6  sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
7  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
8  sec-ch-ua-mobile: ?0
9  Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://127.0.0.1:5000/dashboard
14 Accept-Encoding: gzip, deflate, br
15 Cookie: token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoyLCJlc2VybmFtZSI6ImNoY
   XJtaWUiLCJpcl9hZGlpbiI6ZmFsc2UsImlhdCI6MTc2NDgwMDM2NX0.pvKyUd9GwdweBTt0eoN
   GzFqSarQ_uyKg01GXrFL9VyQ
16 Connection: keep-alive
17
18
```

Response

Pretty Raw Hex Render

```
6  Date: Mon, 08 Dec 2025 00:38:57 GMT
7
8  {
9      "payments":[
10         {
11             "amount":500.0,
12             "biller_name":"Universal Bank Card",
13             "card_number":null,
14             "category_name":"Credit Cards",
15             "created_at":"2025-12-04 00:05:35.607054",
16             "description":"Bill Payment",
17             "id":2,
18             "payment_method":"balance",
19             "processed_at":null,
20             "reference":"BILL1764803135",
21             "status":"pending"
22         },
23         {
24             "amount":120.0,
25             "biller_name":"HealthFirst Insurance",
26             "card_number":null,
27             "category_name":"Insurance",
28             "created_at":"2025-12-03 23:44:10.437478",
29             "description":"Bill Payment",
30             "id":1,
31             "payment_method":"balance",
32             "processed_at":null,
33             "reference":"BILL1764801850",
34             "status":"pending"
35         }
36     ],
37     "status":"success"
38 }
```

4. The ***POST 127.0.0.1:5000/api/virtual-cards/card_id/update-limit*** is vulnerable to BOPLA(Mass assignment). While updating the card limit of a standard card, I managed to add “card_type”: “premium” in the request body instead of just the “card_limit”. The server accepted the request and I successfully changed the limit as well as the card type.

GET 127.0.0.1:5000/api/virtual-cards Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

Auth Type: Be...

Token: ••••••••••••••••••••••••••••••••

The authorization header will be automatically generated when you send the request. Learn more about **Bearer Token** authorization.

Body Cookies Headers (5) Test Results 200 OK 143 ms 893 B Save Response

{} JSON Preview Visualize


```
16  {
17      "balance": 0.0,
18      "card_number": "8068032926666414",
19      "card_type": "standard",
20      "created_at": "2025-12-04 22:28:31.416766",
21      "cvv": "574",
22      "expiry_date": "12/26",
23      "id": 2,
24      "is_active": true,
25      "is_frozen": false,
26      "last_used_at": null,
27      "limit": 1000.0
28  }
```


Runner Start Proxy Cookies Vault Trash

POST 127.0.0.1:5000/api/virtual-cards/2/update-limit Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

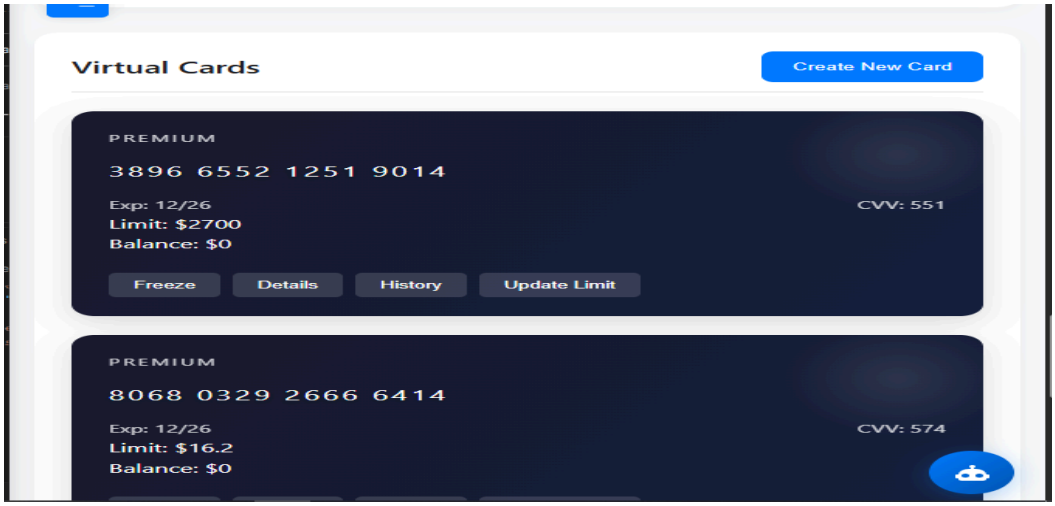

```
1  {
2      "card_limit": 16.2,
3      "card_type": "premium"
4
5
6  }
```


Body Cookies Headers (5) Test Results 200 OK 41 ms 527 B Save Response

{} JSON Preview Visualize


```
6          "current_balance": 0.0,
7          "id": 2,
8          "is_active": true,
9          "is_frozen": false
10     },
11     "updated_fields": [
12         "card_limit",
13         "card_type"
14     ]
15 },
16 "message": "Card updated successfully",
17 "status": "success"
18 }
```


Runner Start Proxy Cookies Vault Trash



Recommendations

The API should filter fields in response and the developer must not rely on UI filtering. Perform checks and validation before accepting data from users and external sources.

Conclusion

APIs play a crucial role in modern application architecture and therefore API Security is very important, the need to identify and fix security issues is paramount. The Vulnerable Bank API project serves as an excellent tool for anyone looking to learn about API vulnerabilities and practice secure coding techniques. By working with Vulnerable Bank API, users can gain practical experience with the OWASP API Top 10 2023 vulnerabilities and become better equipped to secure APIs in real-world applications.